

0.41. Taille 8 : un carré de 17×17

Le même carré qu'au 0.40, avec la taille 8 au lieu de 7 : 17×17 cases autour de (10, 8).

1. Ce qu'il faut comprendre

Un seul changement par rapport au 0.40 : la taille, 4e réglage, passe de 7 à 8.

Ce qui est nouveau ici : la taille 8. La lettre tourne à **8 cases** du centre. La règle $2 \times n + 1$ donne $2 \times 8 + 1 = 17$ cases de côté : un carré de 17×17 .

Où il tombe dans l'écran : autour de (10, 8), de la colonne $10 - 8 = 2$ à $10 + 8 = 18$, et de la ligne $8 - 8 = 0$ à $8 + 8 = 16$. L'écran va de la colonne 0 à 19 et de la ligne 0 à 17 : le carré tient entier.

C'est le plus grand possible. Avec la taille 9, le haut du carré serait à la ligne $8 - 9$, au-dessus de l'écran : il n'existe pas de ligne -1. La taille 8 touche la ligne 0 en haut, la colonne 2 à gauche et la colonne 18 à droite.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <carre>` → [0.33.3. La fonction `carre\(\)` — tourner en carré](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1349 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
carre()         440 o ← main() - une fonction fournie par le langage
main()          38 o ← démarrage - ta fonction
VBlank          8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 4      52 o sur place, dans carre() + AdresseCase (10 o)
effacer() × 3    45 o sur place, dans carre() + AdresseCase (10 o), EffacerCases (12 o)
image() × 4      12 o sur place, dans carre(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
carre() × 1      31 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <carre>      // une tuile qui tourne en carré
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    // Un seul changement par rapport au 0.40 : la taille, 7 → 8.
    //
    //  carre(10, 8, ALPHABET[0], 8, 1, 250, 1);
    //
    //          |
    //          +-- taille 8 : à 8 cases du centre ;
    //              2 × 8 + 1 = 17 cases de côté
    //
    // Le carré, colonnes 2 à 18, lignes 0 à 16 (A : le centre, (10, 8)) :
    //
    // ligne 0 : → → → → → → → → → → → → → → ↓
    // ligne 1 : ↑ . . . . . ↓
    // ligne 2 : ↑ . . . . . ↓
    // ligne 3 : ↑ . . . . . ↓
    // ligne 4 : ↑ . . . . . ↓
    // ligne 5 : ↑ . . . . . ↓
    // ligne 6 : ↑ . . . . . ↓
    // ligne 7 : ↑ . . . . . ↓
    // ligne 8 : ↑ . . . . . A . . . . . ↓
    // ligne 9 : ↑ . . . . . ↓
    // ligne 10 : ↑ . . . . . ↓
    // ligne 11 : ↑ . . . . . ↓
    // ligne 12 : ↑ . . . . . ↓
    // ligne 13 : ↑ . . . . . ↓
    // ligne 14 : ↑ . . . . . ↓
    // ligne 15 : ↑ . . . . . ↓
    // ligne 16 : ↑ ← ← ← ← ← ← ← ← ← ← ← ← ← ←
    carre(10, 8, ALPHABET[0], 8, 1, 250, 1);

    while (true) {
        image();          // le tour est fini : le A reste au centre
    }
}
```

3. Ce qui se passe, étape par étape

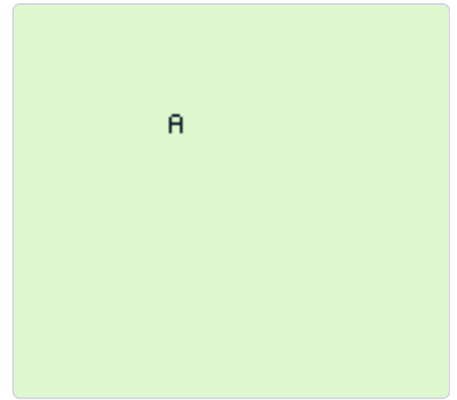
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Un A qui fait un tour en carré de 17×17 cases autour du milieu de l'écran, puis revient au centre.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les quatre coins : (2, 0), (18, 0), (18, 16), (2, 16)

✓ jamais plus loin que la taille 8

✓ sens 1 : du coin, il va à DROITE

✓ le A revient au centre (10, 8) (10,8)

Ensuite : **0.41.1. Taille 8 : un carré de 17×17 — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++